Analysis of the Previous Group's FPGA Matrix-Multiplication Accelerator

Setup, architecture review, bottlenecks and proposed improvements

Kai

September 30, 2026

Abstract

This report documents my analysis of the matrix-multiplication accelerator that the previous project group built for the Zynq UltraScale+ (ZCU102, `xczu9eg-ffvb1156-2-e`) using Vivado/Vitis 2025.1.1. I first describe what was needed to get their project to open and build on my machine. I then walk through the hardware and software architecture, estimate where the clock cycles go, and list the inefficiencies and issues I found. Finally I propose a set of improvements and estimate their effect. My main finding is that the design is dominated by data movement: the actual 5×5 block multiplication takes about 5 cycles, while fetching its operands takes about 100, and loading the matrices from the processor takes longer still. I estimate that the full set of proposed changes could make the end-to-end run roughly 50–100× faster. All numbers in this report are estimates from reading the design; they have not yet been measured on hardware.

Contents

1 Introduction **3**

2 Received files and project setup **3**
- 2.1 Contents of the hand-over . . . 3
- 2.2 Problems encountered during setup . . . 3
- 2.3 Software build . . . 4

3 Architecture of the previous design **4**
- 3.1 Overview . . . 4
- 3.2 Step by step . . . 4
- 3.3 Resource usage and timing . . . 5

4 Cycle budget **5**

5 Findings: inefficiencies and issues **6**
- 5.1 Data movement . . . 6
- 5.2 Compute . . . 6
- 5.3 Robustness and measurement . . . 7

6 Proposed improvements **7**
- 6.1 Quick wins (software only) . . . 7
- 6.2 Small hardware changes . . . 7
- 6.3 Larger architectural changes . . . 8

7 Expected improvement **8**
- 7.1 Comparison with a CPU-only implementation . . . 9

8 Next steps **9**

9 Conclusion **9**

1 Introduction

The goal of the project is to offload matrix multiplication from the ARM processing system (PS) to the programmable logic (PL) of the FPGA in order to accelerate it. The previous group implemented a design that multiplies two 50×50 matrices of 64-bit integers by splitting them into 5×5 blocks. They reported that transferring the matrices between the PS and the PL takes most of the time (roughly 800 of about 900 clock cycles in their measurement).

I received their files via Teams and analysed them with three questions in mind:

1. Can the project be opened and built without errors?
2. How does the design work, and where does the time go?
3. What can be changed to make it faster, and by how much?

2 Received files and project setup

2.1 Contents of the hand-over

File	Contents
`S7_integration.zip` (1.27 GB)	Complete Vivado project (`S7_integration.xpr`), block design `accu_to_DDR`, several exported `.xsa` files.
`S7_integration_vit.zip`	Vitis Unified workspace: platform `interrupt_rerun_plat` and application `interrupt_rerun_test`.
`DDR_INTERFACE_100MHz.zip`	Packaged HLS IP `RAM_INTERFACE`.
`ip_repo_accumulator.zip`	Packaged IP `MatMul5x5_Acc` (accumulator).
`ip_repo_matrix_multiplier.7z`	Packaged IP `MatMul5x5_PAR_IF` (5×5 multiplier).
`Matrix_Partition_Update.zip`	Packaged IP `matrix_partitioner_B`.
`ip_matrix_partition.zip`	Older `matrix_partitioner` (not used in the final design).
`DDR_interface HLS source files`	HLS C++ source of `RAM_INTERFACE`.
`README.MD`	Short notes: use `C:/PLD/s7_integration`, comment out the interrupt, tools are 2025.1.1.

Table 1: Files received from the previous group.

2.2 Problems encountered during setup

1. **Absolute paths.** The Vivado project and the Vitis workspace contain absolute paths (e.g. the platform points to `C:\PLD\S7_integration\interrupt_rerun.xsa`). I therefore used `C:\PLD` as my workspace, with the Vivado project in `C:\PLD\S7_integration` and the Vitis workspace in `C:\PLD\S7_integration_vit`. With this layout the Vitis platform and application built without any changes.

2. **Tool version.** I initially had Vivado 2025.1 installed instead of 2025.1.1. In 2025.1 every IP in the block design (including standard Xilinx IP such as the Zynq PS, SmartConnect and BRAM controllers) was reported as "Newer software version" and locked, and the only offered action was "Remove IP". Installing 2025.1.1 resolved this for 143 of the 145 IPs.

3. **IP repositories.** The project expects IP repositories at `../ip_repo` and `../Cezar_repo` relative to the project. I placed the four custom IPs in `C:\PLD\ip_repo` and created an empty `Cezar_repo` placeholder.

4. **Outdated IP in the hand-over.** Two IPs remain locked with "IP revision change": `RAM_INTERFACE` (design uses Rev. 2114456298, zip contains the older Rev. 2114452861) and `matrix_partitioner_B` (design uses Rev. 3, zip contains Rev. 1). I confirmed this by comparing the zipped sources with the copies Vivado keeps in `S7_integration.gen/.../ipshared` (e.g. the design's `RAM_INTERFACE.vhd` is 178 KB, the zipped one 40 KB). Clicking "Upgrade IP" would therefore *downgrade* the design to an older version. The newest versions are most likely in the previous group's `Cezar_repo`, which was not handed over. Until then the block design can be viewed but not validated or regenerated. The already exported hardware (`interrupt_rerun.xsa` and bitstream) is unaffected and is what the Vitis platform uses.

2.3 Software build

After setup, both the platform and the application build successfully in Vitis 2025.1.1. The remaining warnings in the platform build come from AMD's generated FSBL/PMU firmware and are harmless. I fixed the two warnings in `helloworld.c` (declaration order of `static volatile` and an unused handler parameter).

3 Architecture of the previous design

3.1 Overview

Figure 1 shows the data path as I reconstructed it from the block design `accu_to_DDR`. All user logic runs on a 100 MHz clock (`addn_ui_clkout1` of the PL DDR4 controller); the DDR4 controller itself runs at 300 MHz.

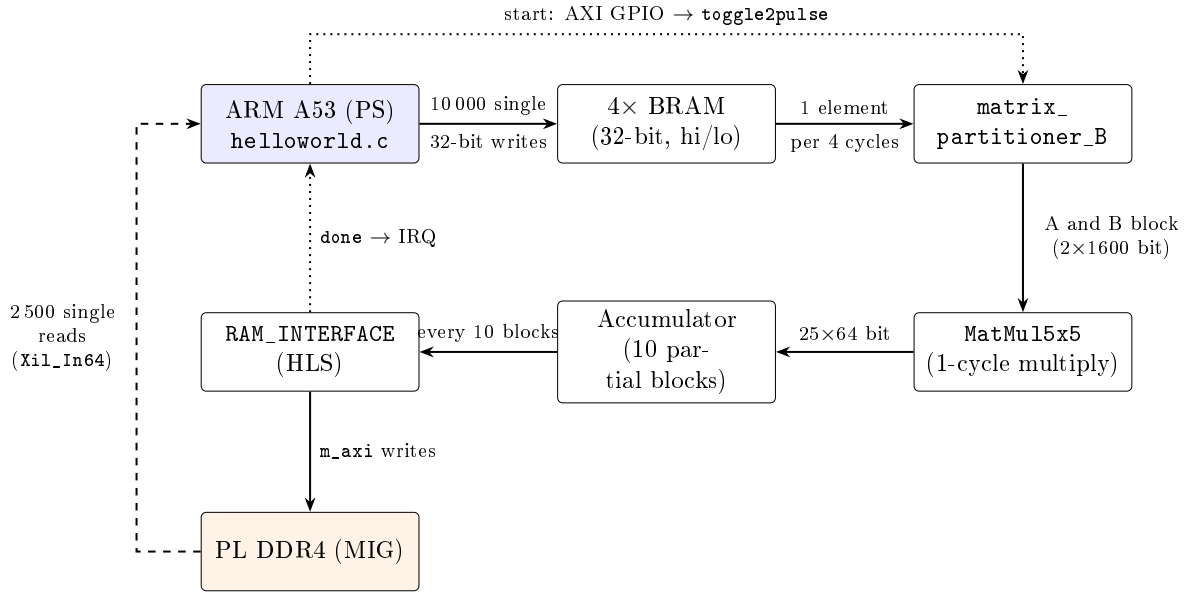


Figure 1: Data path of the previous design (reconstructed from the block design).

3.2 Step by step

1. **Loading (PS).** `helloworld.c` writes matrices A and B (50×50, 64-bit) into block RAM through AXI BRAM controllers (32-bit data width). Each 64-bit element is split into a high and a low 32-bit word stored in two separate BRAMs (`bram_write_64()`). This results in $2 \times 2500 \times 2 = 10\,000$ individual AXI write transactions, each issued by the CPU.

2. **Start.** The PS writes an AXI GPIO output; `toggle2pulse` (pulse width 105 cycles) converts the level change into a start pulse for the partitioner.

3. **Partitioning.** `matrix_partitioner_B` iterates over all block triples (i, j, p) with p fastest ($10 \times 10 \times 10 = 1000$ triples). For each triple it reads block A_{ip} and block B_{pj} element by element from the BRAMs, reassembles the 64-bit values, and outputs two 1600-bit vectors with a valid/ready handshake.

4. **Block multiply.** `MatMul5x5_PAR_IF` wraps `MatMul5x5`, which computes $C = A \times B$ for the 5×5 blocks in a *single* clock cycle (125 signed 64×64-bit multiplications and 128-bit additions). Only the lower 64 bits of each 128-bit result are passed on.

5. **Accumulation.** `MatMul5x5_Accumulator` (`NUM_ACCUMULATIONS` = 10) adds the 10 partial products $A_{ip}B_{pj}$ to form one output block C_{ij}.

6. **Write-back.** The HLS block `RAM_INTERFACE` (free-running, `ap_ctrl_none`) writes each finished 5×5 block to the PL DDR4 at `0x4_0000_0000` via an `m_axi` port, and raises `done` after all 100 output blocks.

7. **Completion and check.** `done` is connected to `pl_ps_irq0` and to a GPIO input. The software waits for the interrupt, computes the reference result on the CPU, and compares it with the result it reads back from PL DDR4 using 2 500 single 64-bit reads.

3.3 Resource usage and timing

From the implementation reports of their last build:

Resource	Used	Available	Utilisation
CLB LUTs	50 779	274 080	18.5 %
CLB registers	80 263	548 160	14.6 %
Block RAM tiles	244	912	26.8 %
DSP slices	1 257	2 520	49.9 %

Table 2: Utilisation after placement (previous group's build).

All timing constraints are met. The 100 MHz user clock has a worst negative slack of +1.40 ns, i.e. the critical path is about 8.6 ns. Half of the chip's DSP slices are used by the single-cycle 5×5 multiplier.

4 Cycle budget

I estimated the cycle counts by reading the RTL state machines (Table 3).

- The partitioner cycles through `ISSUE_ADDR` → `WAIT1` → `WAIT2` → `CAPTURE` for every element, i.e. 4 cycles per element. A block pair has 25 elements (A and B are read in parallel), so one block pair costs about $25 \times 4 = 100$ cycles plus the handshake.
- The multiplier needs about 5 cycles per block pair (accept, load, compute, done, output). Because the partitioner only presents a new block pair every ∼100 cycles, the multiplier is idle about 95 % of the time.
- `RAM_INTERFACE` writes one output block every 10 block pairs (∼1000 cycles), so it is not on the critical path.

This supports the previous group's observation that data movement, not computation, dominates the run time. It holds both on the PS–PL boundary and inside the PL.

Stage	Estimate	Time
Partitioner fetch, per block pair	∼100 cycles	1 µs
Block multiply, per block pair	∼5 cycles	50 ns
Full PL run (1 000 block pairs)	∼102 000 cycles	∼1.0 ms
PS loading (10 000 AXI writes)	–	∼1–2 ms (est.)
PS read-back (2 500 uncached reads)	–	∼0.5–1 ms (est.)
End-to-end		**∼2.5–4 ms**

Table 3: Estimated time budget of the current design at 100 MHz. PS-side times depend on the (unmeasured) latency of a single PS→PL transaction.

5 Findings: inefficiencies and issues

I list the findings roughly in order of their impact on performance.

5.1 Data movement

F1 CPU-driven single-word loading. Every element is written by the CPU as two separate 32-bit AXI transactions (10 000 in total), with no bursts and no DMA. Each transaction crosses the PS–PL interconnect individually. This is almost certainly the largest cost in the system.

F2 Unoptimised software build. The application is compiled with `-O0 -g3` (visible in the build log), which makes the CPU loading loop considerably slower than necessary.

F3 Unnecessary wait states in the partitioner. The BRAMs have a read latency of 1 cycle (no output registers), and the address multipliers (`mult_gen`, constant ×4) have 0 pipeline stages. The `WAIT1` and `WAIT2` states therefore serve no purpose; a new address could be issued every cycle.

F4 One element per cycle at best. Each matrix has only one read port in use, so even without wait states a block needs at least 25 cycles. The 64-bit values are also split over two 32-bit BRAMs, which doubles the number of memories and CPU writes.

F5 No data reuse. Every A block is read 10 times and every B block 10 times: 50 000 element reads for 5 000 unique elements.

F6 Result path through PL DDR4. Results are written to a separate PL DDR4 through a MIG controller (an extra 300 MHz clock domain, clock-domain crossings and significant logic) and then read back by the CPU with 2 500 uncached single reads over the HPM port. `RAM_INTERFACE` writes each 5×5 block with strided addresses, so it can only form short bursts.

5.2 Compute

F7 Oversized, unpipelined multiplier. All 125 64×64-bit multiplications and their additions happen in one combinational cycle. This uses 1 257 DSPs and prevents raising the clock above 100 MHz. Meanwhile the unit is idle about 95 % of the time (F3–F5).

F8 Word width larger than needed. The test data are at most $2\,500^2 \times 50 \approx 3 \times 10^8$, which fits in 32 bits, and the inputs themselves fit in 16 bits. The upper 64 bits of each 128-bit product are discarded anyway.

F9 Signed/unsigned mismatch. `MatMul5x5` interprets the data as `signed`, while the software and the accumulator treat them as unsigned (`uint64_t`, `unsigned`). This is harmless for the small positive test values but gives wrong results for large or negative inputs.

5.3 Robustness and measurement

F10 Interrupt never enabled. In `helloworld.c` the GIC is configured and the handler connected, but `Xil_ExceptionInit()`, `Xil_ExceptionRegisterHandler()` and `Xil_ExceptionEnable()` are never called, and no trigger type is set. The CPU therefore never takes the interrupt and the program hangs at "Waiting for done signal". This matches the README's remark that the interrupt never worked. I have added these calls (the original is kept in `backup/helloworld_original.c`); the fix still has to be verified on the board.

F11 Fragile handshakes. `RAM_INTERFACE` is free-running (`ap_ctrl_none`) and uses plain wires (`ap_none`) for `valid`, `ready` and `done`, with its state kept in `static` variables. This is not a proper valid/ready protocol and can misbehave if `valid` is seen for more than one iteration. AXI-Stream interfaces (`hls::stream`) would make this well defined.

F12 Debug logic left in. Two System ILAs probe very wide buses (up to 1 600 bits). They take block RAM and routing resources and make timing closure harder.

F13 Inflated load measurement. The "BRAM load time" also includes filling the CPU's own reference copies of the matrices (`test_MatrixA/B`), so it overstates the pure transfer time.

F14 Hand-over incomplete. The previous group did not include the newest versions of two IPs, namely `RAM_INTERFACE` and `matrix_partitioner_B`, nor the `Cezar_repo` folder, so the block design cannot currently be regenerated (Section 2.2).

6 Proposed improvements

6.1 Quick wins (software only)

- Build the application with `-O2`.
- Measure the pure BRAM loading separately from filling the reference arrays.
- Time the CPU's own reference multiplication, which the program already performs. This gives the baseline the accelerator has to beat.

6.2 Small hardware changes

- Remove `WAIT1`/`WAIT2` and pipeline the address generation so the partitioner reads one element per cycle ($\sim$4$\times$ faster PL side).
- Replace each pair of 32-bit BRAMs with one 64-bit BRAM (64-bit BRAM controller on the 128-bit HPM0 port). This halves the number of CPU writes and removes the `mult_gen` address multipliers (a constant $\times 4$ is just a shift).
- Remove the ILAs in performance builds.

6.3 Larger architectural changes

- **Let the PL move the data (largest gain).** Add a burst-capable master in the PL (an HLS block with `m_axi` ports, or an AXI DMA) on `S_AXI_HP0_FPD`, reading A and B directly from PS DDR and writing C back to PS DDR. The CPU only flushes/invalidates the caches and starts the transfer. At 128 bits per beat, the 40 KB of input needs about 2 500 beats instead of 10 000 CPU transactions. This also makes the PL DDR4 and its MIG controller unnecessary.
- **Banked memory.** Distribute each matrix over 5 BRAM banks (interleaved by column) so that a full row of a 5×5 block is read in one cycle ($\sim$5 cycles per block instead of 25–100).
- **Data reuse.** Keep a row of A blocks on chip while iterating over the B blocks, or move to a systolic-array organisation, so that each element is fetched from memory once.
- **Narrower, pipelined compute.** Use 32-bit (or 16-bit) inputs if the application allows it and pipeline the multiply/add tree. This reduces DSP usage drastically and allows a 250–300 MHz clock; the freed DSPs can be used for more parallel units.
- **Clean interfaces.** Use AXI-Stream between the blocks and an AXI-Lite control interface (`s_axilite`) instead of GPIO plus `toggle2pulse`.

7 Expected improvement

Table 4 shows my estimate of the effect of each change on the part of the run it affects. Table 5 compares the current and the fully improved design.

Change	Affects	Local speed-up
`-O2` build	PS loading loop	1.5–3×
Remove `WAIT1`/`WAIT2`	PL fetch	$\sim$4× (102k → 26k cycles)
64-bit BRAM instead of 2×32-bit	PS loading	$\sim$2×
Burst transfer (DMA / HLS `m_axi`)	PS loading and read-back	**$\sim$50–100×**
Banked BRAM + data reuse	PL fetch	$\sim$5× (26k → $\sim$5k cycles)
Narrow, pipelined MAC at 250 MHz	PL compute	$\sim$2.5× (clock)
Remove PL DDR4 and ILAs	read-back, timing	enables the above

Table 4: Estimated effect of each proposed change on the stage it affects.

Stage	Current (est.)	After all changes (est.)
Loading A and B	$\sim$1–2 ms	$\sim$15–20 µs
PL computation	$\sim$1.0 ms	$\sim$5–20 µs
Result read-back	$\sim$0.5–1 ms	$\sim$5 µs
Total	**$\sim$2.5–4 ms**	**$\sim$30–50 µs**

Table 5: Estimated end-to-end time before and after all proposed changes.

Overall I expect an end-to-end improvement of roughly **50–100×**. About 70–80 % of that gain comes from a single change: letting the PL fetch and store the data in bursts instead of

the CPU transferring every value. If time allows only one large change, it should be this one, combined with the cheap removal of the partitioner's wait states.

7.1 Comparison with a CPU-only implementation

An important caveat is the baseline. A straightforward 50×50 multiplication (125 000 multiply-accumulates) on the Cortex-A53 compiled with `-O2` should take in the order of 0.1–0.2 ms. By that estimate the *current* accelerator is about 10–30× *slower* than simply using the CPU, and the improved design would be about 3–5× faster than the CPU. For a matrix this small, fixed overheads (cache maintenance, starting the transfer, the interrupt) are a significant part of the total. The advantage of the accelerator grows with the matrix size, because computation scales with n^3 while data transfer scales with n^2.

8 Next steps

1. Obtain the newest `RAM_INTERFACE` and `matrix_partitioner_B` IP (`Cezar_repo`) from the previous group, so the block design can be regenerated.
2. On the board: verify the interrupt fix and run the existing design unchanged.
3. Measure: (a) the latency of a single PS→PL write, (b) the pure loading time, (c) the PL run time, and (d) the CPU-only multiplication time. These replace the estimates in this report with real numbers.
4. Implement the quick wins and the partitioner wait-state removal, and measure again.
5. Design the burst-based data path (PL master on `S_AXI_HP0_FPD`) as the main architectural improvement.

9 Conclusion

The previous group delivered a working hardware data path for blocked 50×50 matrix multiplication on the ZCU102, and I was able to set it up and build it with Vivado/Vitis 2025.1.1. My analysis shows that the design spends the vast majority of its time moving data: the CPU writes every value individually, the partitioner needs 4 cycles per element, blocks are re-read ten times, and results travel through a separate DDR4 and are read back one at a time. The compute unit itself is fast but oversized and idle about 95 % of the time. By letting the PL transfer data in bursts, removing the wait states, banking the memories and narrowing and pipelining the compute, I estimate the run time can be reduced from a few milliseconds to a few tens of microseconds. These estimates now need to be confirmed with measurements on the board.